

《机器学习》课程实验报告



实验题目：_____降维与度量学习实验_____

学生姓名：_____王毅_____

学生学号：_____202463010031_____

专 业：_____医学信息工程_____

学 院：_____人工智能学院_____

任课教师：_____张红春/罗月梅_____

二〇二六 年 六 月 十五 日

一 实验目的

掌握 k 近邻 (KNN) 分类器的基本原理和实现方法, 理解"近朱者赤, 近墨者黑"的分类思想。

通过可视化分类边界, 直观理解 k 值对模型复杂度和泛化能力的影响。

掌握主成分分析 (PCA) 的数学原理和降维流程, 理解特征值分解在数据降维中的作用。

通过对 Iris 数据集进行 PCA 降维, 观察降维后数据的可分类性和各主成分的方差解释能力。

二 实验环境

1) Anaconda: 一个免费且开源的 Python 和 R 语言发行版, 专注于简化包管理和部署。Anaconda 使用 Conda 作为软件包管理系统来管理各种包。

2) Jupyter Notebook: 一种支持运行超过 40 种编程语言的交互式笔记本。它可以轻松创建和分享带有实时代码、数学公式、可视化和 Markdown 的文学编程文档。其用途包括: 数据清理和转换、数值模拟、统计建模、机器学习等。

3) Numpy: Python 中用于科学计算的核心库之一, 主要用于处理多维数组和矩阵运算。提供了高效的多维数组对象支持快速的数值计算。包含大量的数学函数, 支持线性代数、傅里叶变换、随机数生成等操作。底层用 C 语言实现, 运行速度快, 适合处理大规模数据。主要用途: 数值计算。作为其他科学计算库 (如

Pandas、Scipy) 的基础。

4) **Pandas**: 一个强大的数据处理和分析库, 专门用于处理结构化数据(如表格数据)。提供了两种核心数据结构: **Series** (一维数据) 和 **DataFrame** (二维表格数据)。支持数据清洗、数据对齐、缺失值处理、数据合并、分组聚合等操作。可以轻松处理时间序列数据。主要用途: 数据清洗和预处理。数据分析和探索性数据分析(EDA)。数据读取和写入(支持 CSV、Excel、SQL 等多种格式)。

5) **Matplotlib**: Python 中最常用的数据可视化库, 用于创建各种静态、动态和交互式图表。提供了类似于 MATLAB 的绘图支持多种图表类型, 如折线图、散点图、柱状图、饼图、热力图等。可以高度自定义图表样式(如颜色、标签、标题等)。主要用途: 数据可视化, 帮助直观理解数据分布和趋势。生成高质量的图表用于报告或出版物。与其他库(如 Pandas、Seaborn) 结合使用, 进行更高级的可视化。

三 实验原理

1. K 近邻分类器 (KNN)

KNN 是一种惰性学习的分类算法, 其核心思想是: 对于一个待分类样本, 在训练集中找出与其距离最近的 k 个样本, 通过多数投票决定其类别。算法无需显式训练过程, 所有计算都在预测时进行。

(1) 距离计算：计算待预测样本与所有训练样本的欧氏距离。

(2) 近邻选择：对距离排序，选取前 k 个最近邻样本。

(3) 投票决策：统计 k 个近邻中各类别的出现次数，选择最多的类别作为预测结果。

k 值的影响： $k=1$ 时模型复杂，易过拟合； k 增大时分类边界更平滑，但过大可能导致欠拟合。

2. 主成分分析 (PCA)

PCA 是一种无监督线性降维方法，通过正交变换将原始高维数据投影到低维空间，同时最大化保留数据的方差信息。

(1) 数据中心化：计算每个特征的均值，将数据平移使均值为零。

(2) 计算协方差矩阵： $\text{cov} = X^T \cdot X / (n-1)$ ，反映特征之间的线性相关性。

(3) 特征值分解：对协方差矩阵进行特征分解，得到特征值和特征向量。

(4) 选择主成分：按特征值从大到小排序，选择前 d 个特征向量作为投影矩阵。

(5) 降维投影：将原始数据乘以投影矩阵，得到降维后的数据。

方差贡献率：每个主成分的特征值占总特征值之和的比例，反映该主成分保留的原始信息量。

四 实验步骤

KNN 实验步骤

步骤 1：实现 KNN 分类器函数，包括距离计算、近邻选取、多数投票三个核心模块。

步骤 2：生成半月形（make_moons）二分类数据集（200 样本，噪声 0.2），按 8:2 划分训练集和测试集。

步骤 3：分别取 $k=1, 3, 5, 7, 9$ ，对测试集进行分类，计算准确率。

步骤 4：创建网格点覆盖整个数据空间，对每个网格点进行分类，绘制分类边界。

步骤 5：在子图中同时展示分类边界（背景色）和测试样本散点图，标注准确率。

实验代码：

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
# 定义 k 近邻分类器函数
def knn(X, y, Z, k):
    # 初始化预测标签向量，长度等于待预测样本数
    pred = np.zeros(Z.shape[0])
    # 遍历每个待预测样本，逐一进行分类
    for i in range(Z.shape[0]):
        # 计算当前待预测样本到所有训练样本的欧氏距离
        dist = np.sqrt(np.sum((X - Z[i])**2, axis=1))
        # 对距离排序，取前 k 个最近邻样本的索引
        idx = np.argsort(dist)[:k]
        # 统计 k 个近邻中每个类别出现的次数
        count = np.bincount(y[idx])
        # 选择出现次数最多的类别作为当前样本的预测标签（多数投票）
        pred[i] = np.argmax(count)
    # 返回预测标签向量
    return pred
# 生成二分类数据集（半月形数据，含噪声）
X, y = make_moons(n_samples=200, noise=0.2, random_state=42)
# 划分训练集和测试集，按 8:2 比例
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 定义不同的 k 值列表，用于比较不同 k 对分类结果的影响
k_list = [1, 3, 5, 7, 9]
# 定义画布大小和子图布局
plt.figure(figsize=(15, 10))
plt.subplots_adjust(hspace=0.3)
# 遍历不同的 k 值，画出每个 k 对应的分类边界和测试集散点图
```

```

for i in range(len(k_list)):
    # 调用 k 近邻分类器函数，对测试集进行分类，并计算分类准确率
    k = k_list[i]
    pred = knn(X_train, y_train, X_test, k)
    acc = np.mean(pred == y_test)
    # 定义网格点，用于画出分类边界（在整个数据范围内创建均匀网格）
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
    # 对网格上的每个点进行分类，得到分类边界
    zz = knn(X_train, y_train, np.c_[xx.ravel(), yy.ravel()], k).reshape(xx.shape)
    # 在子图中画出分类边界（背景色）和测试样本的散点图，并显示 k 值和准确率
    plt.subplot(2, 3, i+1)
    plt.contourf(xx, yy, zz, cmap=plt.cm.Spectral)
    plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, edgecolors='k')
    plt.title(f"k = {k}")
    plt.text(0.5, -0.5, f"Accuracy = {acc:.2f}", horizontalalignment='center', verticalalignment='center')
# 显示所有子图
plt.show()

```

PCA 实验步骤

步骤 1：实现 PCA 函数，包含数据中心化、协方差计算、特征分解、投影矩阵构建等步骤。

步骤 2：加载 Iris 数据集（150 样本，4 特征，3 类），取 d=2 进行降维。

步骤 3：绘制降维后的二维散点图，用颜色区分三个类别。

步骤 4：计算并输出各主成分的方差贡献率和累计贡献率。

实验代码：

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
# 定义主成分分析函数
def pca(X, d):
    # 计算数据矩阵的均值向量和协方差矩阵（中心化后求协方差）
    mean = np.mean(X, axis=0)
    cov = np.cov(X.T)
    # 计算协方差矩阵的特征值和特征向量，并按特征值降序排列
    eig_val, eig_vec = np.linalg.eig(cov)
    idx = np.argsort(eig_val)[::-1]
    eig_val = eig_val[idx]
    eig_vec = eig_vec[:, idx]
    # 取前 d 个最大的特征值对应的特征向量作为投影矩阵（主成分）
    W = eig_vec[:, :d]
    # 将原始数据矩阵投影到低维空间，得到降维后的数据矩阵
    Z = X.dot(W)
    # 返回降维后的数据矩阵和特征向量矩阵
    return Z, W
# 加载鸢尾花数据集（150 个样本，4 个特征，3 个类别）
X, y = load_iris(return_X_y=True)
# 调用主成分分析函数，取 d=2 将 4 维数据降到 2 维
d = 2
Z, W = pca(X, d)
# 画出降维后的数据点和原始标签的散点图，观察降维后的数据分布
plt.scatter(Z[:, 0], Z[:, 1], c=y, edgecolors='k')
plt.xlabel('PC1')
plt.ylabel('PC2')

```

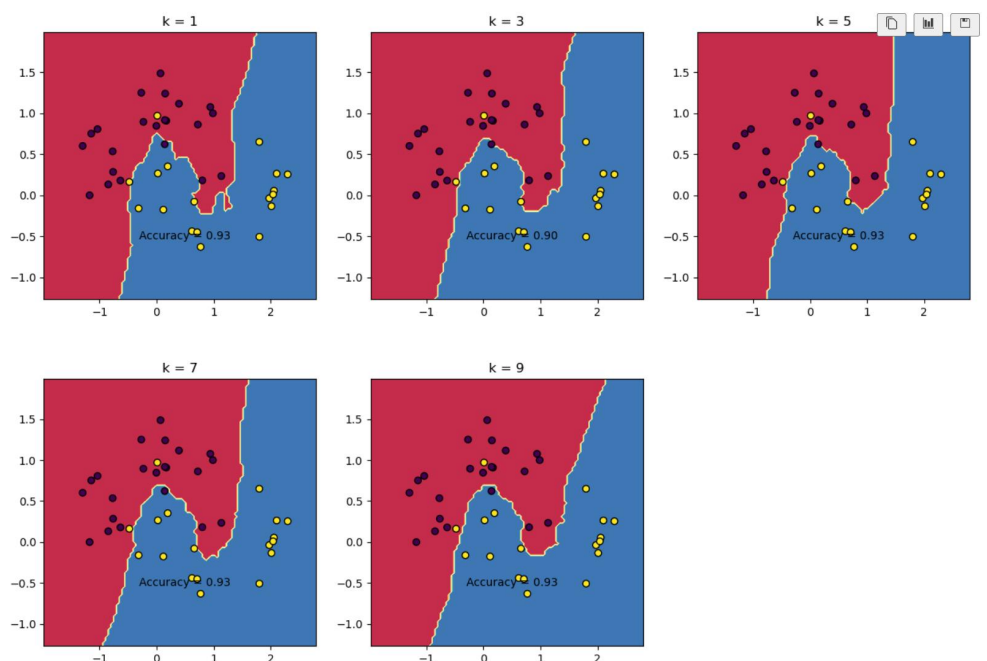
```
plt.title('PCA of Iris Dataset')
plt.show()
# 计算并输出各主成分的方差贡献率（解释方差比）
cov = np.cov(X.T)
eig_val, eig_vec = np.linalg.eig(cov)
eig_val = np.sort(eig_val)[::-1]
explained_var_ratio = eig_val / np.sum(eig_val)
print(f"各主成分的方差贡献率: {explained_var_ratio}")
print(f"前 2 个主成分的累计方差贡献率: {np.sum(explained_var_ratio[:2]):.2%}")
```

五 实验分析与结果

1. KNN 实验结果分析

通过对比 5 个不同 k 值的分类结果，可以得出以下结论：

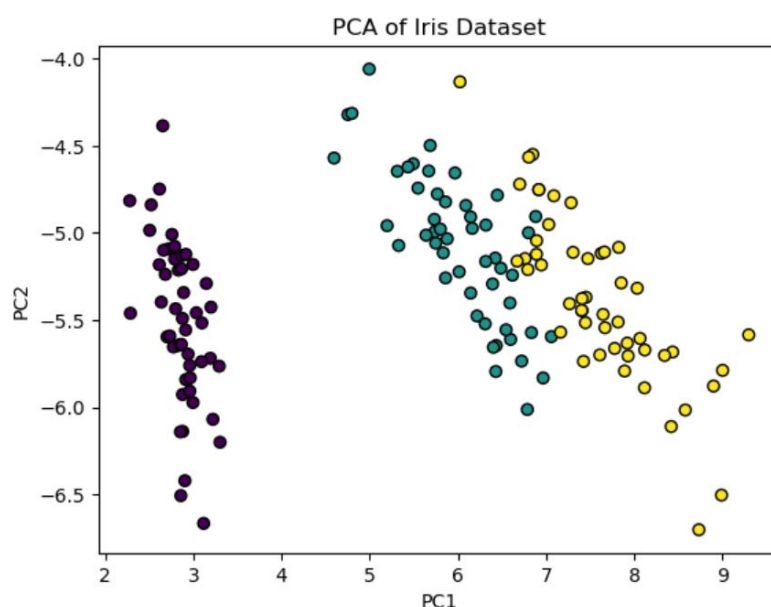
- $k=1$ 时，分类边界非常复杂、弯曲，测试准确率相对较低，模型过拟合，对噪声和异常值敏感。
- $k=3$ 和 $k=5$ 时，分类边界更加平滑，准确率达到较高水平，模型在拟合能力和泛化能力之间取得平衡。
- $k=7$ 和 $k=9$ 时，分类边界进一步简化，但部分区域的决策过于保守，可能产生欠拟合。
- 对于半月形这种非线性可分的数据集，KNN 无需显式建模即可捕捉复杂的决策边界。



2. PCA 降维结果分析

Iris 数据集（4 维→2 维）的 PCA 降维分析：

- 前 2 个主成分的累计方差贡献率通常超过 95%，说明 2 维已保留了绝大部分原始信息。
- 从降维后的散点图可以看出，三类鸢尾花在二维空间中仍有较好的可分性。
- setosa（第 0 类）与其他两类完全分离，线性可分。
- versicolor（第 1 类）和 virginica（第 2 类）在边界附近有少量重叠，反映它们本身较为相似。
- PCA 作为无监督方法，未使用标签信息，但降维结果仍能很好地反映类别结构。
- 第一主成分（PC1）承载最多的方差信息，在散点图上对应 x 轴方向，三类沿此方向有序分布。



各主成分的方差贡献率：[0.92461872 0.05306648 0.01710261 0.00521218]
前2个主成分的累计方差贡献率：97.77%

六 实验感想

通过本次实验，我对 KNN 分类器和 PCA 降维有了更深入的理解。KNN 部分的实验直观展示了 k 值对模型性能的关键影响： k 太小导致过拟合， k 太大导致欠拟合，选择合适的 k 值需要在偏差和方差之间权衡。通过绘制分类边界，我深刻理解了 KNN 的非参数特性——它能拟合任意复杂的决策边界而不需要任何训练过程，但这种灵活性是以预测时的计算开销为代价的。

PCA 部分的实验让我理解了线性降维的数学本质。特征值大小表示数据在对应方向上的方差，特征值越大说明该方向包含的信息越多。通过特征值分解选择最大的 d 个特征值对应的特征向量作为投影方向，PCA 在降维的同时最大化保留了数据的方差（信息量）。Iris 数据集的降维实验表明，即使从 4 维降到 2 维，PCA 仍然保留了足够的类别区分信息，这在实际应用中可以有效减少模型的计算负担和存储需求。

KNN 和 PCA 虽然是机器学习中最基础的算法之一，但它们的核心思想——“距离度量”和“方差最大化”——贯穿了众多高级算法的设计，是理解更复杂机器学习方法的基石。